

Skill 3 · Base de Conversão LSP → Java

Versão: v1.3 · Interna · `skill-03-base-conversao-lsp-java.md`

Skill interna — **não** é fluxo de usuário. Aplique as regras globais do Router. Em conflito de assinatura, **revalide na Skill 2 / página oficial**.

Fronteira: Skill 2 = links/alias; **esta skill** = mecânica + catálogo + padrões de compilação + exemplos.
Precedência: docs oficiais Skill 2 → catálogo oficial abaixo → padrões `padrao_compilacao` (Eclipse) → anexos sanitizados → `validacao_manual` .
Crescimento do catálogo confirmada : só com doc oficial / Skill 2. Achados de compilação entram como `padrao_compilacao` , não como inventados.

Como consultar (ordem obrigatória)

- A. Restrições + Regra de ouro + Invariantes ← nunca pular
- B. Workflow (6 passos)
- C. Tipos / sintaxe / VaPara / Funcao (se necessário)
- D. Catálogo – só a família do item LSP
- E. Acesso a dados (USU_* / R* / ContextSession) se houver cursor/SQL
- F. Armadilhas Eclipse + checklist
- G. Exemplos – só o padrão análogo
- H. Saída tipada para a Skill 1

Não leia o catálogo inteiro de ponta a ponta. Localize a **família** no índice. Em conflito de assinatura, **revalide na Skill 2**.

Índice rápido do arquivo

Precisa de...	Vá para
Proibições / minutos / Sem Situacao.getMinutos	Restrições absolutas
Modelo mental LSP→contexto	Regra de ouro
Passo a passo	Workflow
Tipagem / Se→if / VaPara / Funcao	Tipos e sintaxe
Esqueleto Apuracao / Consistência / BH	Esqueletos + tipos de regra
DatPro, HorSit, FPxMar, históricos...	Catálogo → família
TipCon / MarcacaoRegra / overloads HorSit	Catálogo + notas padrao_compilacao
Cursor USU_* / .sc / ContextSession	Acesso a dados
Erros Eclipse / anti-padrões	Armadilhas
CalculaQtdMinutos / interjornada / TipCon SQL	Exemplos sanitizados
Campos LSP conflitantes	Tabela conflitantes

Fontes oficiais

Fonte	URL	Uso
Equivalência das funções de regras	https://documentacao.senior.com.br/gestao-de-pessoas-hcm/6.10.4/informacoes-adicionais/rotinas/gpo/integracao-controle-ponto-refeitorio/equivalencia-funcoes-regras.htm	Mapa oficial
Índice das Funções HCM 6.10.4	https://documentacao.senior.com.br/gestao-de-pessoas-hcm/6.10.4/customizacoes/funcoes.htm	Assinatura + contexto do método

Catálogo abaixo: evidência `confirmada` (salvo nota). Exemplos: `padrao_anexo` .

Assinatura/contexto incompletos no catálogo → abrir Índice das Funções (Skill 2) antes de gerar código.

Quando usar / não usar

Usar	Não usar
Converter/mapear variáveis e funções de apuração LSP→Java	Só mentoria; inventar método; só citar link (Skill 2)

Restrições absolutas

1. Preferir o catálogo; se faltar → Skill 2 → `validacao_manual` (não inventar).
2. Horas em APIs de ponto = **minutos inteiros** (`int` , não `double` solto).
3. SQL/cursor → API semântica do catálogo **antes** de `EntitySession/ContextSession` — exceto tabela **USU_* custom** sem API (aí `ICursor+ .sc`).
4. **Proibido** `getSituacao(...).getMinutos()` / `setMinutos(...)` — use `getHorSit` / `setHorSit` / `zeraHorasSituacao` .
5. Sanitize nomes de cliente em exemplos anexados; package do projeto é **parâmetro** (ex. sanitizado `gestaopontoCustom`) — nunca vaziar cliente.
6. Sem `Mensagem()` /popup em apuração — preferir `mensagemLog` .
7. Confirmar se o método existe no **contexto** da regra (apuração ≠ consistência ≠ BH ≠ geral).
8. `execute()` **sem** `throws Exception` ; auxiliares `private` no **nível da classe** (nunca aninhados).
9. `ContextSession/DBCenter` da plataforma ≠ Senior SQL 2 (SQL 2 continua proibido).

Regra de ouro

LSP e Java diferem no **modelo de execução**: variáveis/funções globais LSP → **métodos no objeto de contexto**.

Quem só troca `Inicio/Fim` por `{ }` produz código que não compila.

Workflow (6 passos)

1. Identificar contexto (apuração / consistência / bloqueio / fechamento BH / geral).
2. Inventariar construções LSP.
3. Mapear no catálogo (nome e ordem de parâmetros).
4. Traduzir mecânica (getters/setters, arrays→métodos, cursor→ `List` , `End` →retorno).
5. Traduzir sintaxe (`Se` → `if` , `=` comparação→ `==` / `.isEqual()` , etc.).

6. Revisar armadilhas.

Invariantes

1. Toda variável/função de contexto LSP → chamada de método.
2. Não inventar assinatura.
3. Conferir ordem de parâmetros.
4. Horas = minutos inteiros.
5. Tipagem forte Java.
6. Sem popup em apuração.
7. Arrays indexados → métodos/coleções (não `getApuDiu(1)` inventado).
8. Método deve existir no **contexto** da regra.

Tipos e sintaxe

LSP	Java
Numero inteiro	<code>int / long</code>
Numero decimal	<code>double</code>
Alfa	<code>String</code>
lógico 0/1	<code>boolean</code>
Data	<code>LocalDate</code> (ou padrão do SDK — não misturar Joda + <code>java.time</code> sem necessidade)
Hora (minutos)	<code>int minutos</code>
marcação	<code>MarcacaoRegra</code> (<code>padrao_compilacao</code> ; preferir a <code>Marcacao</code> genérica)
Se / Senao / Enquanto / Inicio/Fim	<code>if / else / while / { }</code>
Vapara / labels	<code>early return; break/continue; ou flag + do { } while(false)</code>
Funcao sem end	<code>private void</code> no nível da classe
Funcao com 1 end	retorno tipado do método
Funcao com vários end	<code>int[] / objeto</code> (não closures aninhadas)
Numero[n] array	<code>int[n+1]</code> ou <code>double[n+1]</code> (LSP 1-based → Java 0-based)
Tabela	<code>List<record></code> / classe interna
@ ... @	<code>//</code>
= comparação	<code>== / .isEqual() / .equals()</code>
<>	<code>!=</code>
decimal ,	<code>.</code>

Instruções

1. Seguir “Como consultar” (A→H)
2. Identificar contexto
3. Buscar item só na família do Catálogo
4. Assinatura/contexto duvidosos → Índice das Funções (Skill 2)
5. Ausente → validacao_manual

Tipos de regra e esqueletos (padrao_compilacao)

Tipo LSP	Classe base	Contexto	
Apuração	custom.senior.apuracao.Apuracao	getContextoApuracao()	Regra
Consistência	custom.senior.apuracao.ConsistenciaAcertos	getContextoConsistenciaAcerto()	Regra
Manutenção BH	custom.senior.bancohoras.RegraManutencaoBH	getContextoManutencaoBH()	Regra
Fechamento BH	custom.senior.bancohoras.RegraFechamentoBH	getContextoFechamentoBH()	Regra

Package: o informado pelo usuário/projeto. Exemplo sanitizado: `gestaopontoCustom` (não `gestaoDoPontoCustom`).

Apuração

```
@Rule(description = "Regra de Apuracao")
public class RegraApuracao extends Apuracao {
    @Override
    public void execute() { // SEM throws Exception
        ContextoGeralRH ctxGeral = getContainer().getContextoGeral();
        ContextoApuracao ctx = getContainer().getContextoApuracao();
    }

    private void auxiliar(ContextoApuracao ctx, int param) { /* nível da classe */ }
}
```

Consistência de Acertos

```
@Rule(description = "Consistencia de Acertos")
public class RegraConsistencia extends ConsistenciaAcertos {
    @Override
    public void execute() {
        ContextoConsistenciaAcerto ctx =
getContainer().getContextoConsistenciaAcerto();
        // extras: getHorSitAnterior, getDataInicial/Final, getHorSitAnteriorFaixa
    }
}
```

Fechamento BH

```
@Rule(description = "Fechamento BH")
public class RegraFechamentoBH extends custom.senior.bancohoras.RegraFechamentoBH {
    @Override
    public void execute() {
        ContextoFechamentoBH ctx = getContainer().getContextoFechamentoBH();
    }
}
```

Imports frequentes (padrao_compilacao): java.time.LocalDate / LocalDateTime , com.senior.rule.Rule , com.senior.dataset.ICursor , MappedParamProvider , Colaborador , MarcacaoRegra , TipoIntervalo , TipoHoraExtra , ContextSession , DBCenter , IResultSet , com.senior.dataset.IEntity (+ @Entity / @Field), readonly IR030EMP / IR060DSI /...

Catálogo de equivalência (oficial Senior 6.10.4)

Fonte: [Equivalência das funções de regras](#).

Métodos tipicamente em contextoApuracao / container — **confirmar contexto e assinatura** no Índice das Funções.

Famílias (âncora de busca)

Família	Itens típicos LSP
Data / sistema	DatPro, HorSis, DiaSem, DatIni...
Situações	HorSit[], SitAnt[], TotSit[], MotSit
Horas apuradas/previstas	ApuDiu[], ApuNot[], TraDiu, PrvTra[]...
Marcações	FPxMar, FLeMar, DatMar, QtdMar...
Escala / horário / feriado	EscAtu, CodHor, FerFil...
Históricos / usuário	CarEmp, CodSinEmp, CodUsu...
Compensação / BH / log	TemCmp[], RetBHRDat, MensagemLog...

Data / sistema

LSP (Controle de Ponto)	Java (Gestão do Ponto)
DatPro (implícito via Ano/Dia/MesPro)	getData()
AnoPro	getAnoData(Date data) / getData()
MesPro	getMesData(Date data)
DiaPro	getDiaData(Date data)
AnoSis / MesSis / DiaSis	getAnoData / getMesData / getDiaData

HorSis / ExtSis	Nativo em Java
DatIni / Datfim	getDataInicial() / getDataFinal()
DiaSem, DiaDom...DiaSab	Preferir getDiaSem(Date); alt. getData().getDayOfWeek().getValue() (padrao_compilacao)
MesAtu	getMesData(Date data)
RetornaAnoData / RetornaDiaData / RetornaMesData	getAnoData / getDiaData / getMesData

Situações / HorSit

LSP	Java
HorSit[]	getHorSit(int) / getHorSit(LocalDate, int), setHorSit, somaHorasSituacao, zeraHorasSituacao, zeraHorasSituacaoFaixa, getHorSitFaixa — overloads padrao_compilacao: getHorSit(int[]), zeraHorasSituacao(varargs)
SitAnt[]	getHorSitAnterior(int) / getHorSitAnteriorFaixa (consistência)
TotSit[] / BuscaTotalizadoresSituacoes	getTotalSituacoes(int codigoTotalizador, date data) (e overload com intervalo)
MotSit	getMotivoAcerto(int situacao)

Apuração diurna/noturna / trabalhadas / previstas

LSP	Java
ApuDiu[] / ApuNot[]	getHorasSeparadas(...) / getHoras(Marcacao, Marcacao, ...)
HrtRaD[] / HrtRan[] / TraDiu / TraNot	getHorasTrabalhadas(int parte)
MinPvd[] / MinPvn[] / PrvTra[] / PrvTrd / PrvTrn / VprVho / VprVin / VprVtr	getTotalMinutosPrevisto(...) / getHorasPrevistas(...) / getTotalMinutosPrevistoProrrogado(...)
FatRed	getHorarioPrevistoColaborador(...)
ExtrasIntervalo	getExtrasIntervalo / getExtrasIntervaloAnterior / getExtrasIntervaloPosterior
MinExD[] / MinExn[] / MsaInd[] / MsaInn[] / IniExt / FimExt / PerExt / PriExt / QtdExt / TipExt / DiaExt	getIntervalosCalculados() / getIntervaloCalculado(int indice)

Marcações

LSP	Java
-----	------

FPxMar / FLeMar / ...	List<MarcacaoRegra> getMarcacoesRealizadas(boolean) — getData() → .toLocalDate(); índice Java 0-based
QtdMar / TotMar	Preferir getQtdMarcacoesRealizadas(boolean); alt. .size() (padrao_compilacao)
DulMar / HulMar	getMarcacaoAnterior() → tipo MarcacaoAnterior: só diferencaMinutos(data, hora) — sem getHora/getData
MinJor	Preferir getHorasInterjornadaRealizada(); alt. diferencaMinutos se só MarcacaoAnterior
MinMJo	getHorasInterjornadaPrevista()
TipCon	Não existe Colaborador.getTipCon() — SQL SELECT TIPCON FROM R034FUN ... via ContextSession (padrao_compilacao)
ApuDiu/ApuNot tipados	Também getHorasSeparadas(TipoIntervalo TipoHoraExtra) → ISeparacaoHoras
log / pendência / alerta	mensagemLog, setGerarPendencia, criarAlerta (try/catch)

Escala / horário / filial / feriado

LSP	Java
EscAtu	getEscala()
EscEmp / RetEscEmp	getHistoricoEscala()
EscTrf	getEscalaHistorico()
TemTes	getTrocaEscala(Date data)
TemThr	getTrocaHorario(Date data)
RetornaEscala	getEscalaPrevistaColaborador(...)
CodHor / RetornaHorarioApurado	getHorario() / getHorario().getCodigo() — especiais 9996–9999 (folga/feriado/comp/DSR)
HorEsc / HorTrf	getHorarioEscala()
HorDFe	getHorarioOriginalEscala()
HorFol	getCodigoHorarioFolga() / getHorarioFolga()
HorPfo	getHorarioProjecaoFolga()
RetornaHorario / RetornaBatidaHorario	getHorarioPrevistoColaborador(...)
NumPer	getNumeroPeriodos(int codHor)
NumInt	getNumeroIntervalos()
NinRef	getNumeroIntervaloRefeicao()

TurInt	getTurmaIntervalo()
RetMinRefHTr	getMinutosRefeicaoPrevisto()
FilEmp / RetFilEmp / DatAltFil / EmpAltFil	getHistoricoFilial()
FerFil	getFeriadoFilial(Date data)
VerDatFer	getFeriado(Date data)

Históricos colaborador / usuário

LSP	Java
CarEmp / EstCarEmp / RetCarEmp	getHistoricoCargo()
CcuEmp / DatAltCcu / RetCcuEmp	getHistoricoCentrodeCusto()
CodSinEmp / RetSinEmp	getHistoricoSindicato()
CodVinEmp / RetVinEmp	getHistoricoVinculo()
LocEmp / RetLocEmp	getHistoricoLocal(...)
CodAfs / IniAfs / FimAfs / TemAfs / QtdAfs[]	getHistoricosAfastamento()
BusCraTit	getHistoricosCracha() / getHistoricosCrachaProvisorio()
RetApuPon	getHistoricoApuracao(colaborador, data)
CodUsu	getUsuarioAtivo()
NomUsu	getUsuario(long codigoUsuario)
RetornaDesGrupo / RetornaQtdGrupos	getGrupos(long codigoUsuario)
AssociaUsuColab	associarUsuarioColaborador(...)
RetColabPorCodUsu	getUsuarioColaborador(...)

Compensação / banco de horas / cálculo

LSP	Java
TemCmp[] / DtICmp[] / DtFCmp[] / PerCmp[] / QtdCmp[] / SitCmD[] / SitCmN[] / TipCmp[]	getCompensacoes() / getCompensacoes(LocalDate data)
LimBa1 / LimBa2	getEscala()
RetBHRDat	getSaldoBanco(int banco, int empresa, int tipo, int cadastro, LocalDate data)
TipCal	getTipoCalculo()
MensagemLog	mensagemLog(String mensagem)

VerificaAbrangenciaNumero	getAbrangencia(string abrangencia, int numero)
RetornaCodLoc / RetornaNumLoc / RetNivLoc	getCodigoLocal / getNumeroLocal / getQtdNiveisLocal

Definição de situações (padrão complementar)

Cursor R0145IN / R030EMP para CodDsi → preferir API (padrao_anexo até confirmar na doc):

```
int codDsi = contextoApuracao.getDefinicaoSituacoes().getCodigo();
```

Campos LSP conflitantes (padrao_compilacao)

LSP	Erro comum	Correto
TipCon	col.getTipCon()	SQL R034FUN.TIPCON
nMar / marcações	índice 1-based / tipo errado	List<MarcacaoRegra> 0-based + .toLocalDate()
MarcacaoAnterior	getHora() / getData()	só diferencaMinutos
HorSit[n]	tratar como array Java	get/setHorSit(codSit, minutos)
SitAnt	getter inventado	getHorSitAnterior no contexto de consistência
QtdMar	inventar campo	API oficial ou .size()

Domínio HCM — constantes úteis

Valor	Uso
CodHor 9996–9999	Folga / feriado / compensado / DSR (<9996 = horário normal)
1320 / 300	Janela noturna 22h–05h (minutos)
660 / 360	Limites CLT interjornada / sem intervalo
1440	Virada de dia em CalculaQtdMinutos

Acesso a dados (após API-first)

Tabela USU_* custom?

SIM → IEntity + .sc + ICursor (EntitySession)

NÃO (R* nativa / USU_* em tabela nativa) → ContextSession SELECT ou DBCenter DML
Readonly em com.senior.rh.entities.readonly.*? → importar, não recriar

Fatal: .sc apontando para Rxxxxx → "Tabela ... inválida. É permitido mapear somente tabelas de usuário".

ICursor (USU_*)

```

MappedParamProvider params = new MappedParamProvider();
params.setParam("vNumEmp", vNumEmp);
ICursor<IMinhaTabela> cur =
getContainer().getEntitySession().newCursor(IMinhaTabela.class);
cur.addFilter("USU_NumEmp = :vNumEmp", params);
cur.open();
try {
    if (cur.first()) { IMinhaTabela row = cur.read(); }
} finally { cur.close(); }

```

Alternativas: `EntitySessionProvider.getSession()` ; `CursorUtil + SearchMode.EXACT_MATCH` ; `setOrder + OrderDirection` .

ContextSession (SELECT) vs DBCenter (DML)

	ContextSession	DBCenter
Uso	SELECT	INSERT/UPDATE/DELETE
Close	Não fechar (container)	Fechar no finally
Placeholder	?	?
Índice ResultSet	base-0 (getInt(0) = 1º campo)	—
databaseId típico	—	ex. "vetorh" (confirmar projeto)

Interface + .sc (só USU_*)

- `import com.senior.dataset IEntity` (**nunca** `com.senior.g5...IEntity`)
- `@Entity + @Field` ; IDs de usuário em `long` ; flags `char 'S'`
- `.sc` = JSON puro começando com `{ ; id` = nome do arquivo sem extensão; `query: from USU_...`
- Null: `isXNull()` antes de ler

Readonly do framework (importar)

`IR030EMP` , `IR060DSI` , `IR004HOR` , `IR010SIT` , `IR010TOB` , `IR014SIN` , `IR070ACC` , ...

Armadilhas práticas

Armadilha / erro Eclipse	Correção
Só trocar sintaxe sem mapear variáveis	Inventário + getters/setters primeiro
Copiar ordem de parâmetros LSP	Confirmar no Índice das Funções
<code>getSituacao().get/setMinutos</code>	<code>getHorSit / setHorSit / zeraHorasSituacao</code>
Cursor SQL por reflexo	Buscar família no catálogo; <code>USU_*</code> → <code>ICursor</code>
<code>getHorSit(variavelDeMinutos)</code>	1º arg = código da situação
Inventar <code>getApuDiu(1)</code>	<code>getHoras / getHorasSeparadas</code>

execute() throws Exception	Remover throws (IRulePoint)
Método aninhado / código solto	Auxiliares no nível da classe
col.getTipCon()	SQL R034FUN.TIPCON
MarcacaoAnterior.getHora/getData	diferencaMinutos
LocalDateTime → LocalDate	.toLocalDate() em MarcacaoRegra.getData()
.sc em tabela R*	Remover .sc; ContextSession
.sc não-JSON / BOM	Reescrever começando com {
import com.senior.g5...IEntity	com.senior.dataset.IEntity
Package gestaoDoPontoCustom	Package do projeto (ex. gestaopontoCustom)
ResultSet base-1	base-0
Filename ≠ classe pública	Renomear arquivo (Windows: temp rename)
Inventar método	validacao_manual
Misturar java.time e Joda	Preferir java.time / SDK

Exemplos sanitizados (use só o análogo)

HorSis + setHorSit

```
Se (HorSis > 870) Inicio
  HorSit = 120;   @ situação 100 @
Fim;
```

```
java.time.LocalTime agora = java.time.LocalTime.now();
int minutos = agora.getHour() * 60 + agora.getMinute();
if (minutos > 870) {
    contextoApuracao.setHorSit(100, 120);
}
```

HorSit — errado vs certo

```
// ERRADO
contexto.getSituacao(xNorAnt).setMinutos(0);
// CERTO
contexto.zeraHorasSituacao(xNorAnt);
int v = contexto.getHorSit(xExDDsr); // minutos int
contexto.setHorSit(xExDDsr, v + vNorDes);
```

Marcações FLeMar/FPxMar → List

```
for (MarcacaoRegra m : contextoApuracao.getMarcacoesRealizadas(false)) {
    LocalDate d = m.getData().toLocalDate();
    if (d.isAfter(contextoApuracao.getData())) { /* ... */ }
}
```

End → retorno (RetBHRDat)

```
// ordem Java ≠ LSP – confirmar no índice
int saldo = contextoApuracao.getSaldoBanco(codbhr, numemp, tipcol, numcad,
    new LocalDate(2014, 9, 24));
```

Múltiplos End → objeto

```
private ResultadoDiurnoNoturno calculaDiurnoNoturno(...) {
    return new ResultadoDiurnoNoturno(minutosDiurnos, minutosNoturnos);
}
```

Troca de situação

```
private int trocarSituacao(ContextoApuracao ctx, int origem, int destino, int
minutosPendentes) {
    int minutosOrigem = ctx.getHorSit(origem);
    if (minutosOrigem >= minutosPendentes && minutosOrigem > 0 && minutosPendentes >
0) {
        ctx.setHorSit(destino, minutosPendentes);
        ctx.setHorSit(origem, minutosOrigem - minutosPendentes);
        return 0;
    }
    if (minutosOrigem < minutosPendentes && minutosOrigem > 0 && minutosPendentes >
0) {
        ctx.setHorSit(destino, minutosOrigem);
        ctx.setHorSit(origem, 0);
        return minutosPendentes - minutosOrigem;
    }
    return minutosPendentes;
}
```

CalculaQtdMinutos (virada de dia)

```
private int calculaQtdMinutos(LocalDate d1, int horIni, LocalDate d2, int horFim) {
    if (d1.isBefore(d2)) horFim = horFim + 1440;
    return horFim - horIni;
}
```

TipCon via ContextSession

```

int nTipCon = 0;
try {
    IResultSet rs = ContextSession.getSession().executeQuery(
        "SELECT TIPCON FROM R034FUN WHERE NUMEMP = ? AND TIPCOL = ? AND NUMCAD = ?",
        numEmp, tipCol, numCad);
    if (rs.next()) nTipCon = rs.getInt(0);
} catch (Exception ex) {
    ctx.mensagemLog("Erro TipCon: " + ex.getMessage());
}

```

Interjornada (MarcacaoAnterior)

```

if (ctx.getMarcacaoAnterior() != null) {
    int n = ctx.getMarcacaoAnterior().diferencaMinutos(datMar, horaMar);
    if (n < 0) n = -n;
    // preferir getHorasInterjornada* quando disponível no contexto
}

```

EntitySession / ICursor (USU_* sem API)

```

ICursor<IEntidadeCustom> cursor = getContainer().getEntitySession()
    .newCursor(IEntidadeCustom.class);
try {
    cursor.addFilter("USU_Campo = :v", new MappedParamProvider("v", valor));
    cursor.open();
    if (cursor.first()) { /* read */ }
} finally {
    cursor.close();
}

```

Justificar ausência de API semântica; emitir interface+ `.sc` só para `USU_*`.

Saída para a Skill 1

```

contexto: Apuracao | FechamentoBH | outro | indefinido
item_lsp: ...
equivalente_java: ...
evidencia: confirmada | padrao_compilacao | padrao_anexo | validacao_manual
fonte: equivalencia-funcoes-regras | indice-funcoes | compilacao | anexo
limite: ...

```

Relacionados

Skill 2 (URLs/aliases) · Skill 1 · Skill 5 (`CHK-SITAPI` , `CHK-THROWS` , `CHK-SCNAT` , `CHK-TIPCON` , `CHK-MARANT` , ...)